

SRE CASE STUDY

AZURE ARC HYBRID CLOUD LAB

A Proof-of-Value project demonstrating hybrid cloud orchestration, observability, FinOps, Policy as Code, and Infrastructure as Code using K3s Kubernetes and Microsoft Azure Arc.

Author: Bartosz Suszko

Company: IT Solutions Bartosz Suszko, Olsztyn, Poland

Date: May 2026

GitHub: github.com/buth11/sre-homelab-azure-arc

Stack: K3s v1.35.4 | Azure Arc | Prometheus | Grafana | Terraform | KQL | Azure Policy

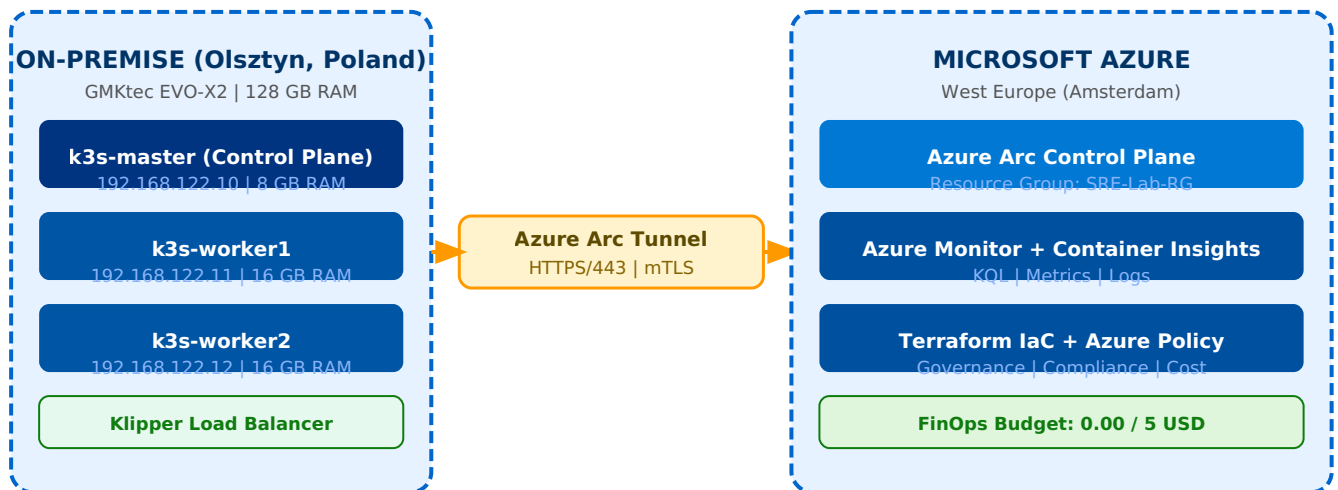
Table of Contents

1. Business Context and Architecture
2. On-Premise Infrastructure — K3s Cluster
3. FinOps — Cloud Cost Control
4. Hybrid Integration — Azure Arc
5. Observability — Container Insights
6. KQL Queries — Diagnostics and SLO
7. Load Balancing — Round-Robin
8. Prometheus and Grafana
9. Infrastructure as Code — Terraform
10. Policy as Code — Azure Policy
11. GitHub — Documentation
12. Postmortem — POST-001 etcd Split-Brain
13. Results and Conclusions
14. Troubleshooting Log

Chapter 1: Business Context and Hybrid Architecture

This project demonstrates design, deployment, and operation of a production-grade hybrid environment connecting on-premises infrastructure with Microsoft Azure, using the same technology stack found in enterprise SaaS deployments.

Architecture Diagram



Layer	Technology	Purpose
Hypervisor	KVM on Fedora 43	Node virtualization
OS	Ubuntu Server 24.04 LTS	All K3s nodes
Kubernetes	K3s v1.35.4+k3s1	Lightweight K8s distribution
Cloud Bridge	Azure Arc (agent v1.34.2)	Hybrid control plane
Cloud Monitoring	Azure Monitor + Container Insights	Telemetry and KQL analytics
Local Monitoring	Prometheus + Grafana v3.21	Open-source observability stack
IaC	Terraform ~3.100	Infrastructure as Code
Governance	Azure Policy	Policy as Code
Package Manager	Helm v3.21.0	K8s application deployment
Load Balancer	K3s Klipper LB	Layer 4 round-robin routing
FinOps	Azure Cost Management	Budget guardrail: 5 USD/month

Chapter 2: On-Premise Infrastructure

Node	Role	IP	CPU	RAM	Disk
k3s-master	Control Plane	192.168.122.10	4 vCPU	8 GB	40 GB
k3s-worker1	Worker	192.168.122.11	4 vCPU	16 GB	60 GB
k3s-worker2	Worker	192.168.122.12	4 vCPU	16 GB	60 GB

Chapter 3: FinOps

The screenshot displays the Microsoft Azure Cost Management console for user Bartosz Suszko. The left sidebar contains navigation links for resource management, billing, and monitoring. The main area shows the 'Budżety' (Budgets) section with a table listing the 'SRE-Lab-Budget'.

Nazwa	Zakres	Resetuj okres	Data utworzenia	Data wygaśnięcia	Budżet	Prognostyczne	Oszacowane wy...	Postęp
SRE-Lab-Budget	55df28b8-39ed-58b...	Monthly	1.05.2026	30.04.2028	5,00 USD	0	\$0.00	0.00%

Screenshot 1: Azure Cost Management - SRE-Lab-Budget 5 USD/month, 0.00 USD final cost.

Chapter 4: Hybrid Integration — Azure Arc

The screenshot shows the Microsoft Azure portal interface. The main content area displays the 'Azure Arc | Klastry Kubernetes' page for a cluster named 'K3s-Homelab'. The cluster is connected and running version v1.35.4+k3s1. The left sidebar shows the Azure navigation menu, and the right sidebar shows the cluster's properties and monitoring options.

Cluster Details:

- Subskrypcja:** Azure subscription 1
- Identyfikator subskrypcji:** b023e8b8-4445-4d4f-9fcf-d20a6c1fa093
- Grupa zasobów:** SRE-Lab-RG
- Stan:** Connected
- Lokalizacja:** West Europe
- Menedżer floty:** K3s
- Wersja agenta:** 1.33.0
- Wersja platformy Kubernetes:** 1.35.4+k3s1
- Łączna liczba węzłów:** 3
- Łączna liczba rdzeni:** 12

Monitoring and Alerts:

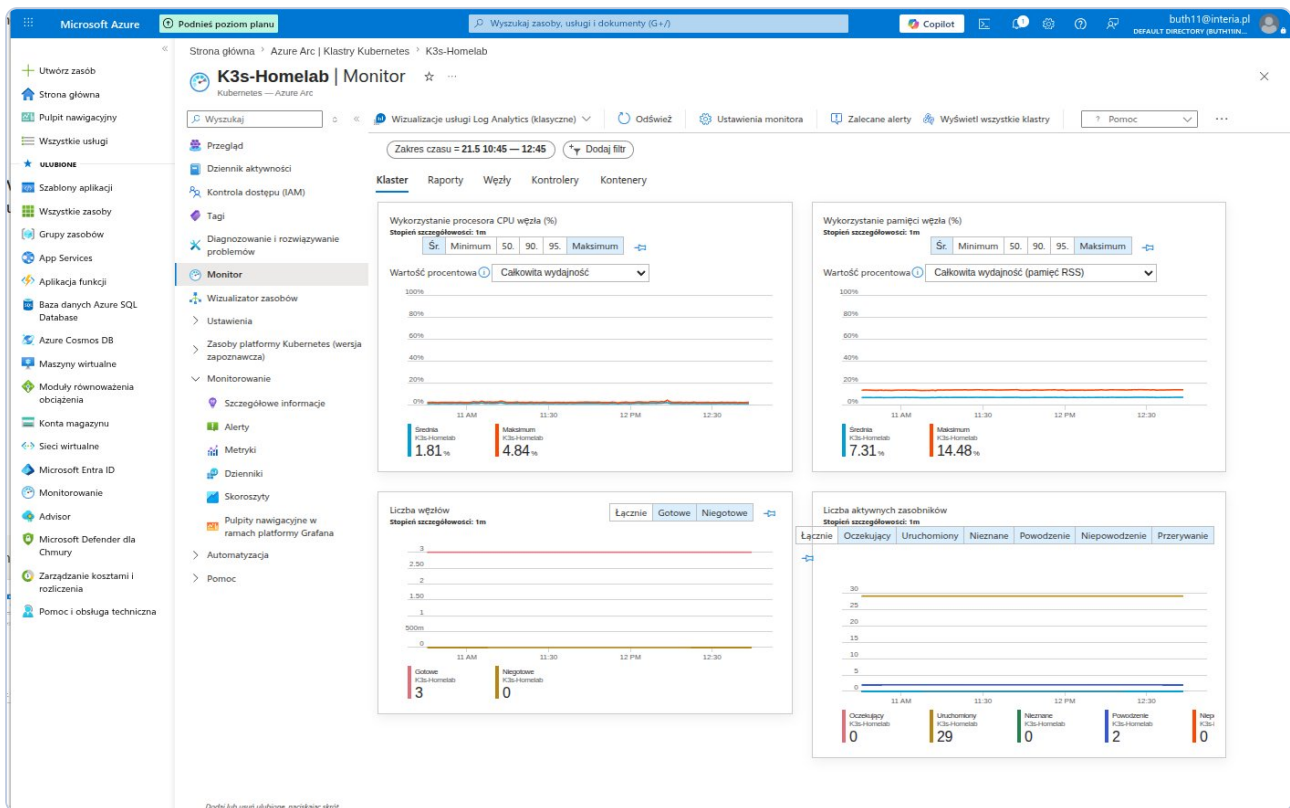
- Alerty:** Włącz diagnostykę peryskopu dla tego klastra
- Tagi:** Tagi (edit), Dodaj tagi
- Właściwości:** Właściwości, Monitorowanie, Możliwości, Samouczki

Screenshot 2: Azure Arc - K3s-Homelab Connected, v1.35.4+k3s1, 3 nodes, 12 cores.

Chapter 5: Observability

```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × Desktop: bash × (buth11) 192.168.122.10 ×
{"principalId": "a3fe4a8c-7a2e-4b99-8de9-2150d617cb76",
 "tenantId": null,
 "type": "SystemAssigned"
},
"isSystemExtension": false,
"name": "azuremonitor-containers",
"packageUri": null,
"plan": null,
"provisioningState": "Succeeded",
"releaseTrain": "Stable",
"resourceGroup": "SRE-Lab-RG",
"scope": {
  "cluster": {
    "releaseNamespace": "azuremonitor-containers"
  },
  "namespace": null
},
"statuses": [],
"systemData": {
  "createdAt": "2026-05-22T07:44:41.936626+00:00",
  "createdBy": null,
  "createdByType": null,
  "lastModifiedAt": "2026-05-22T07:44:41.936626+00:00",
  "lastModifiedBy": null,
  "lastModifiedByType": null
},
"type": "Microsoft.KubernetesConfiguration/extensions",
"version": null
}
buth11@k3s-master:~$ sudo k3s kubectl get pods -n kube-system | grep ama
[sudo] password for buth11:
ama-logs-9m9x9          3/3      Running    0          37s
ama-logs-9scpr          3/3      Running    0          37s
ama-logs-rs-65b58c7f96-f8lds  2/2      Running    0          37s
ama-logs-spcdp          3/3      Running    0          37s
buth11@k3s-master:~$
```

Screenshot 3: AMA pods Running (4 agents, 0 restarts, 37s after install).



Screenshot 4: Azure Monitor - CPU avg 1.58%/max 2.30%, RAM avg 6.27%/max 12.01%, 3 nodes Ready, 29 pods.

Chapter 6: KQL

The screenshot displays the Microsoft Azure portal interface. On the left is the navigation pane with various service categories. The main area is titled 'Log Analytics workspace' and 'DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU'. A KQL query is entered in the 'New Query 1' editor:

```
1 KubeNodeInventory
2 where TimeGenerated > ago(1h)
3 summarize arg_max(TimeGenerated, *) by Computer
4 project
5     Computer,
6     Status,
7     KubeletVersion
8 order by Computer asc
```

The query results are displayed in a table with columns: Computer, Status, and KubeletVersion. The results show 3 nodes in a 'Ready' state with version 'v1.35.4+k3s1'.

Computer	Status	KubeletVersion
k3s-master	Ready	v1.35.4+k3s1
k3s-worker1	Ready	v1.35.4+k3s1
k3s-worker2	Ready	v1.35.4+k3s1

KQL-1: 3 nodes Ready, v1.35.4+k3s1.

The screenshot shows the Microsoft Azure portal interface. On the left is the navigation pane with options like 'Create a resource', 'Dashboard', and 'All services'. The main area displays the 'Log Analytics workspace' for 'DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU'. A KQL query is entered in the 'New Query' box:

```
1 KubePodInventory
2 | where TimeGenerated > ago(1h)
3 | summarize arg_max(TimeGenerated, *) by Name, Namespace
4 | summarize PodCount = count() by Namespace, PodStatus
5 | order by Namespace asc, PodCount desc
```

The results are displayed in a table with columns: Namespace, PodStatus, and PodCount.

Namespace	PodStatus	PodCount
azure-arc	Running	12
default	Running	3
kube-system	Running	14
kube-system	Succeeded	2

KQL-2: azure-arc 12 Running, default 3 Running, kube-system 14 Running + 2 Succeeded.

The screenshot shows the same Microsoft Azure portal interface. The KQL query in the 'New Query' box is:

```
1 KubeEvents
2 | where TimeGenerated > ago(24h)
3 | project TimeGenerated, Namespace, Name, Reason, Message
4 | order by TimeGenerated desc
5 | take 20
```

The results are displayed in a table with columns: TimeGenerated [UTC], Namespace, Name, Reason, and Message.

TimeGenerated [UTC]	Namespace	Name	Reason
5/22/2026, 7:44:56.000 AM	kube-system	ama-logs-9scpr	FailedMount
5/22/2026, 7:44:56.000 AM	kube-system	ama-logs-rs-65b58c7996-f8ld5	FailedMount
5/22/2026, 7:35:32.000 AM	azure-arc	kube-aad-proxy-5f59465457-l2...	Unhealthy
5/22/2026, 7:35:25.000 AM	azure-arc	config-agent-75548f59c5-k4zrv	Unhealthy
5/22/2026, 7:35:13.000 AM	k3s-worker2		Rebooted
5/22/2026, 7:35:11.000 AM	k3s-worker1		Rebooted
5/22/2026, 7:35:09.000 AM	kube-system	metrics-server-786d997795-zz...	Unhealthy
5/22/2026, 7:35:07.000 AM	k3s-master		Rebooted
5/21/2026, 10:53:39.000 AM	kube-system	ama-logs-89qgj	Unhealthy
5/21/2026, 10:52:38.000 AM	kube-system	ama-logs-rs-65b58c7996-nlxhw	Unhealthy
5/21/2026, 8:37:39.000 AM	kube-system	svclb-aras-demo-app-6f6f7352...	FailedScheduling
5/21/2026, 8:37:39.000 AM	kube-system	svclb-aras-demo-app-6f6f7352...	FailedScheduling
5/21/2026, 8:37:39.000 AM	kube-system	svclb-aras-demo-app-6f6f7352...	FailedScheduling

The message for the last event is: "0/3 nodes are available: 1 node(s) didn't have free ports for the requested pod ports, 2 node(s) didn't satisfy ... s are available: 1 node(s) didn't have free ports for the requested pod ports, 2 Preemption is not helpful for sc..."

KQL-3: 13 events 24h - Rebooted, Unhealthy (temporary), FailedScheduling.

Microsoft Azure | Upgrade | Search resources, services, and docs (G+/)

Home > Log Analytics workspaces > DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU

Log Analytics workspace

Default Directory (buth11@interia.pl@microsoft.com)

Log Analytics workspace

Search

New Query 1

Time range: Set in query | Show: 1000 results | KQL mode

```

1 KubePodInventory
2 | where TimeGenerated > ago(24h)
3 | summarize arg_max(TimeGenerated, *) by Name, Namespace
4 | where PodRestartCount > 0
5 | project Namespace, Name, PodRestartCount, PodStatus
6 | order by PodRestartCount desc

```

Namespace	Name	PodRestartCount	PodStatus
azure-arc	extension-manager-85899588c...	3	Running
azure-arc	clusterconnect-agent-5bc68597...	3	Running
kube-system	svcib-traffic-4b7f5cb6-88kx...	2	Running
azure-arc	extension-events-collector-6c9...	2	Running
azure-arc	clusteridentityoperator-767768...	2	Running
azure-arc	kube-aad-proxy-5f53465457-42...	2	Running
kube-system	svcib-traffic-4b7f5cb6-c2gs...	2	Running
azure-arc	config-agent-75548f5c5-k4zrv...	2	Running
azure-arc	cluster-metadata-operator-8db...	2	Running
kube-system	svcib-traffic-4b7f5cb6-9587...	2	Running
azure-arc	metrics-agent-6d9947664-rcr44...	2	Running
azure-arc	controller-manager-7cb948bdf...	2	Running
azure-arc	resource-sync-agent-54d88c5...	2	Running
azure-arc	flux-logs-agent-768c454d-9g...	1	Running
azure-arc	logcollector-8bd4468b4-4s2zw...	1	Running
kube-system	ama-logs-e6g5p...	1	Running
kube-system	helm-install-traffic-ggts...	1	Succeeded
kube-system	svcib-aras-demo-app-8581479...	1	Running
default	aras-demo-app-f78659546-vb...	1	Running
kube-system	coredns-c4dffb5f-84kx...	1	Running
kube-system	local-path-provisioner-5c4dc5...	1	Running
kube-system	metrics-server-786d97795-zz...	1	Running
kube-system	svcib-aras-demo-app-8581479...	1	Running

0s 381ms | Display time (UTC+00:00) | Query details | 1 - 24 of 27

KQL-4: 27 pods with restarts after VM reboot - all Running.

Microsoft Azure | Upgrade | Search resources, services, and docs (G+/)

Home > Log Analytics workspaces > DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU

Log Analytics workspace

Default Directory (buth11@interia.pl@microsoft.com)

Log Analytics workspace

Search

New Query 1

Time range: Last 24 hours | Show: 1000 results | KQL mode

```

1 KubeNodeInventory
2 | where TimeGenerated > ago(24h)
3 | summarize
4 |   TotalSamples = count(),
5 |   ReadySamples = countif(Status == "Ready")
6 | by Computer
7 | extend AvailabilityPct = round(toreal(ReadySamples) / toreal(TotalSamples) * 100, 2)
8 | project Computer, AvailabilityPct, ReadySamples, TotalSamples
9 | order by Computer asc

```

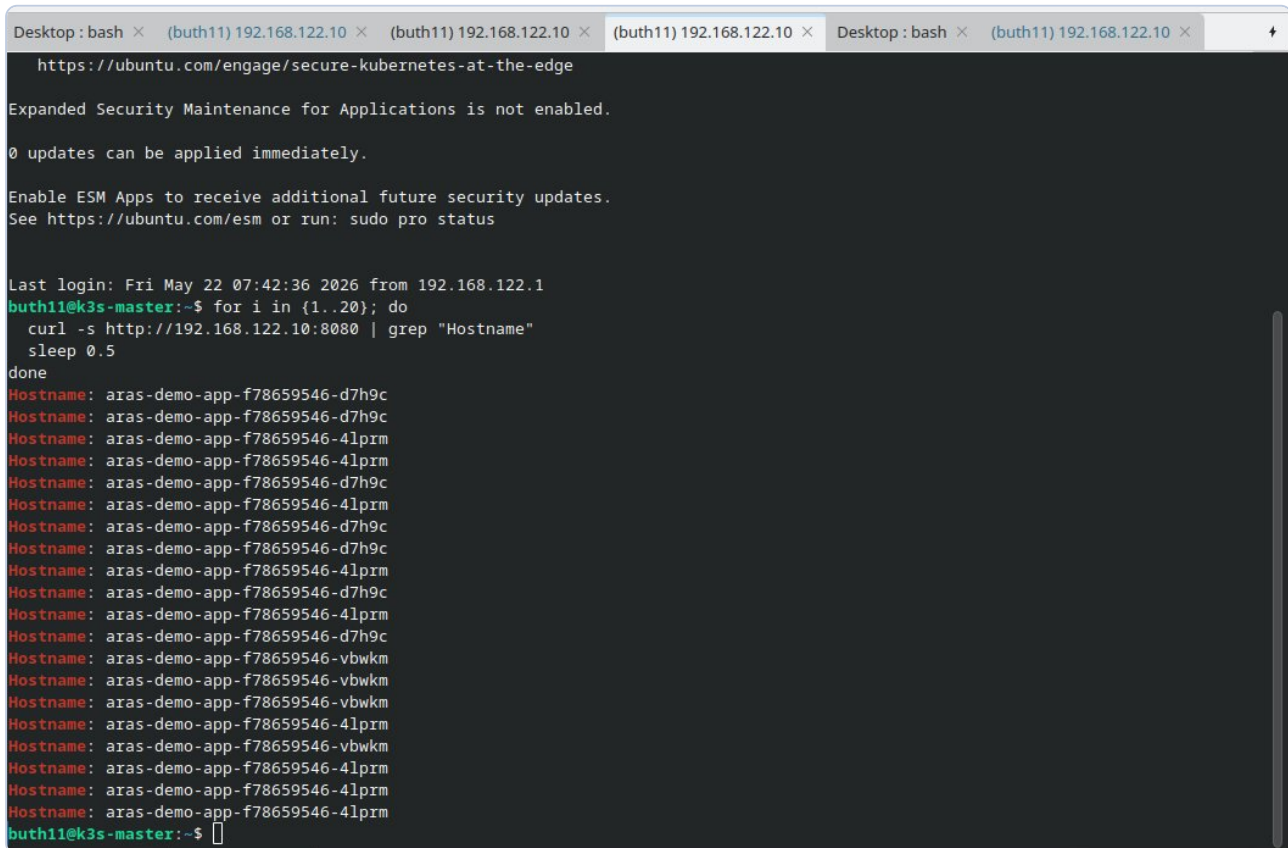
Computer	AvailabilityPct	ReadySamples	TotalSamples
k3s-master	100	184	184
k3s-worker1	100	184	184
k3s-worker2	100	184	184

0s 582ms | Display time (UTC+00:00) | Query details | 1 - 3 of 3

KQL-5: SLO 100% - all 3 nodes 184/184 samples Ready.

SLO: 100% availability in 24h window. Enterprise target: 99.9% (43 min error budget/month). Error budget intact.

Chapter 7: Load Balancing

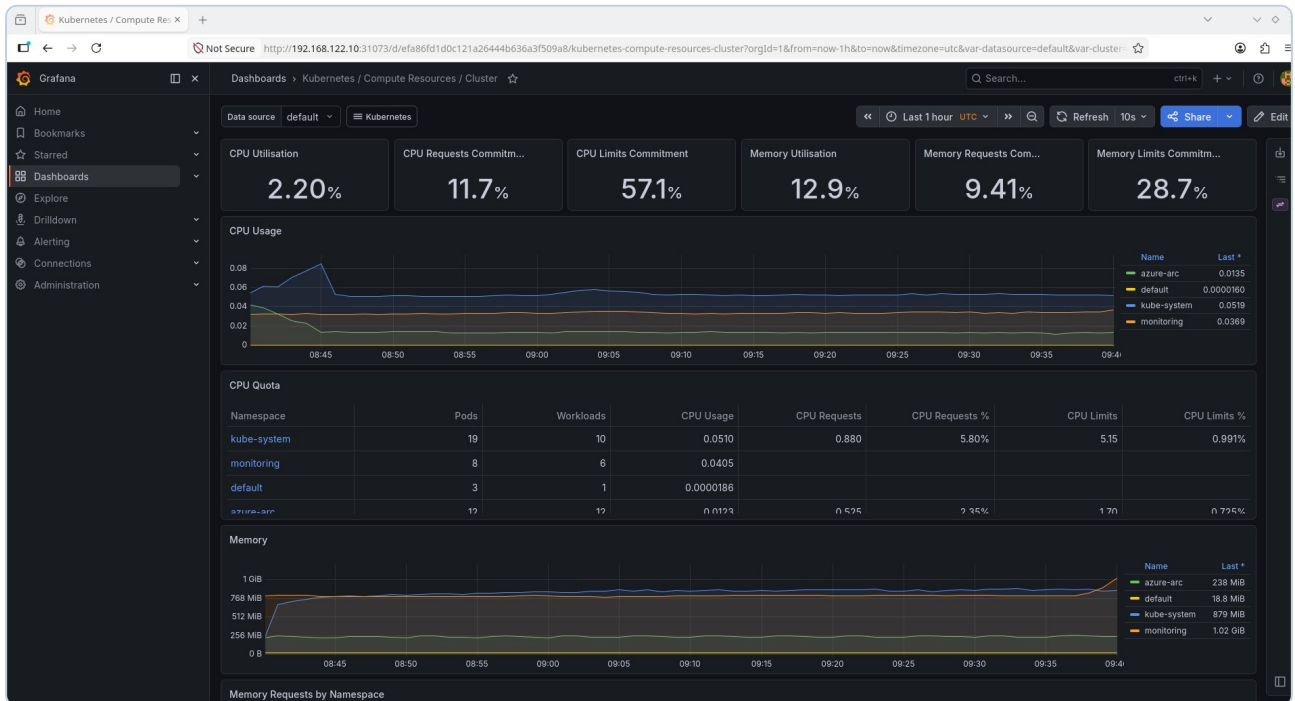


The screenshot shows a terminal window with a dark background. At the top, there are several browser tabs and a terminal window. The terminal output shows a message about Ubuntu security updates, followed by a login message. Then, a script is executed: `for i in {1..20}; do curl -s http://192.168.122.10:8080 | grep "Hostname" sleep 0.5 done`. The output of the script is a list of 20 hostnames, alternating between three different pod names: `aras-demo-app-f78659546-d7h9c`, `aras-demo-app-f78659546-4lprm`, and `aras-demo-app-f78659546-vbwkm`. The terminal ends with a prompt `buth11@k3s-master:~$`.

```
Desktop : bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × Desktop : bash × (buth11) 192.168.122.10 ×  
https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
Expanded Security Maintenance for Applications is not enabled.  
0 updates can be applied immediately.  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
Last login: Fri May 22 07:42:36 2026 from 192.168.122.1  
buth11@k3s-master:~$ for i in {1..20}; do  
  curl -s http://192.168.122.10:8080 | grep "Hostname"  
  sleep 0.5  
done  
Hostname: aras-demo-app-f78659546-d7h9c  
Hostname: aras-demo-app-f78659546-d7h9c  
Hostname: aras-demo-app-f78659546-4lprm  
Hostname: aras-demo-app-f78659546-4lprm  
Hostname: aras-demo-app-f78659546-d7h9c  
Hostname: aras-demo-app-f78659546-4lprm  
Hostname: aras-demo-app-f78659546-d7h9c  
Hostname: aras-demo-app-f78659546-4lprm  
Hostname: aras-demo-app-f78659546-d7h9c  
Hostname: aras-demo-app-f78659546-4lprm  
Hostname: aras-demo-app-f78659546-d7h9c  
Hostname: aras-demo-app-f78659546-4lprm  
Hostname: aras-demo-app-f78659546-d7h9c  
Hostname: aras-demo-app-f78659546-vbwkm  
Hostname: aras-demo-app-f78659546-vbwkm  
Hostname: aras-demo-app-f78659546-vbwkm  
Hostname: aras-demo-app-f78659546-4lprm  
Hostname: aras-demo-app-f78659546-vbwkm  
Hostname: aras-demo-app-f78659546-4lprm  
Hostname: aras-demo-app-f78659546-4lprm  
Hostname: aras-demo-app-f78659546-4lprm  
buth11@k3s-master:~$
```

Screenshot 5: Round-robin - 20 requests across 3 pods, 900 parallel requests, zero errors.

Chapter 8: Prometheus & Grafana



Screenshot 6: Grafana - CPU 2.20%, Memory 12.9%, CPU Quota per namespace.

Chapter 9: Terraform

```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × terraform: bash × (buth11) 192.168.122.10 ×
Plan: 6 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ action_group_id           = (known after apply)
+ budget_name               = "SRE-Lab-Budget"
+ log_analytics_workspace_id = (known after apply)
+ log_analytics_workspace_key = (sensitive value)
+ resource_group_id         = (known after apply)
+ resource_group_name        = "SRE-Lab-RG"
+ subscription_id           = "b023e8b8-4446-4d4f-9fcf-d20a6c1fa093"

Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
  terraform apply "tfplan"
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$ terraform apply tfplan
random_string.suffix: Creating...
random_string.suffix: Creation complete after 0s [id=5plwbb]
azurerm_resource_group.sre_lab: Creating...
azurerm_consumption_budget_subscription.sre_lab: Creating...
azurerm_consumption_budget_subscription.sre_lab: Still creating... [00m10s elapsed]
azurerm_consumption_budget_subscription.sre_lab: Creation complete after 19s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/providers/Microsoft.Consumption/budgets/SRE-Lab-Budget]

Error: A resource with the ID "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG" already exists -
to be managed via Terraform this resource needs to be imported into the State. Please see the resource documentation for "azure
rm_resource_group" for more information.

with azurerm_resource_group.sre_lab,
on main.tf line 26, in resource "azurerm_resource_group" "sre_lab":
26: resource "azurerm_resource_group" "sre_lab" {

buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$
```

Terraform Plan: 6 resources to create, 0 to change, 0 to destroy.

```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × terraform: bash × (buth11) 192.168.122.10 ×

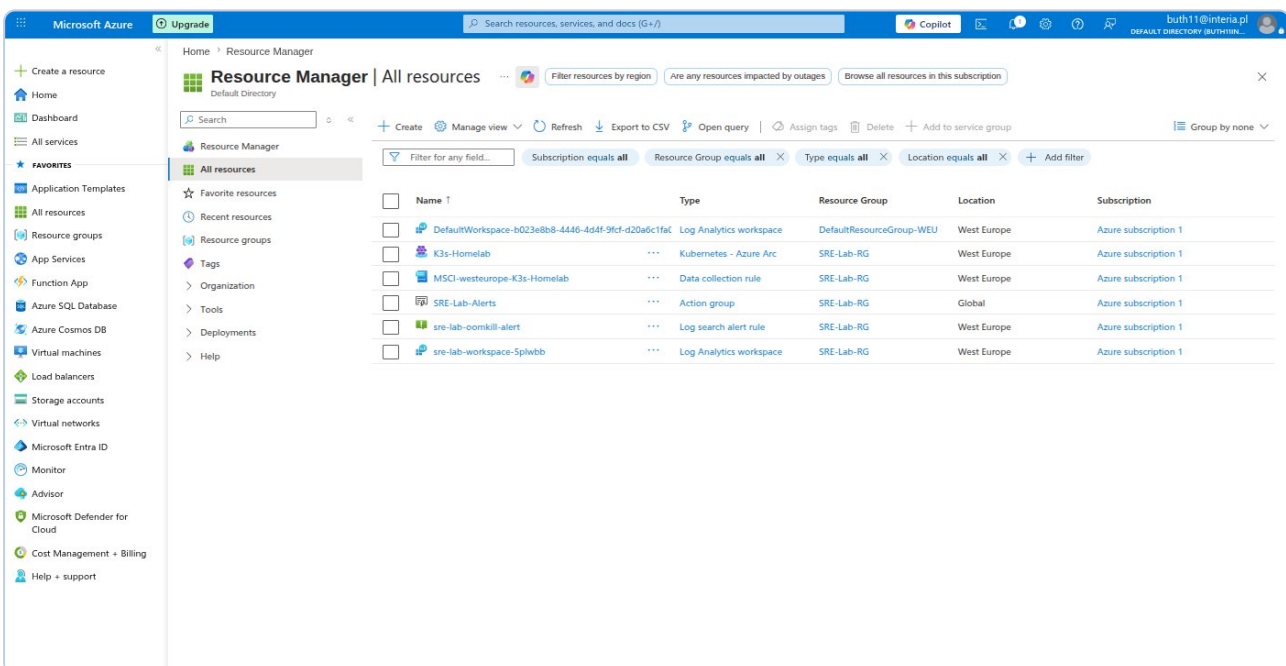
Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
terraform apply "tfplan"
azurerm_resource_group.sre_lab: Modifying... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG]
azurerm_resource_group.sre_lab: Modifications complete after 3s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG]
azurerm_monitor_action_group.sre_alerts: Creating...
azurerm_log_analytics_workspace.sre_lab: Creating...
azurerm_monitor_action_group.sre_alerts: Creation complete after 4s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/actionGroups/SRE-Lab-Alerts]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m10s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m20s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m30s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m40s elapsed]
azurerm_log_analytics_workspace.sre_lab: Creation complete after 47s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.OperationalInsights/workspaces/sre-lab-workspace-5plwbb]
azurerm_monitor_scheduled_query_rules_alert_v2.oom_kill: Creating...
azurerm_monitor_scheduled_query_rules_alert_v2.oom_kill: Creation complete after 5s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/scheduledQueryRules/sre-lab-oomkill-alert]

Apply complete! Resources: 3 added, 1 changed, 0 destroyed.

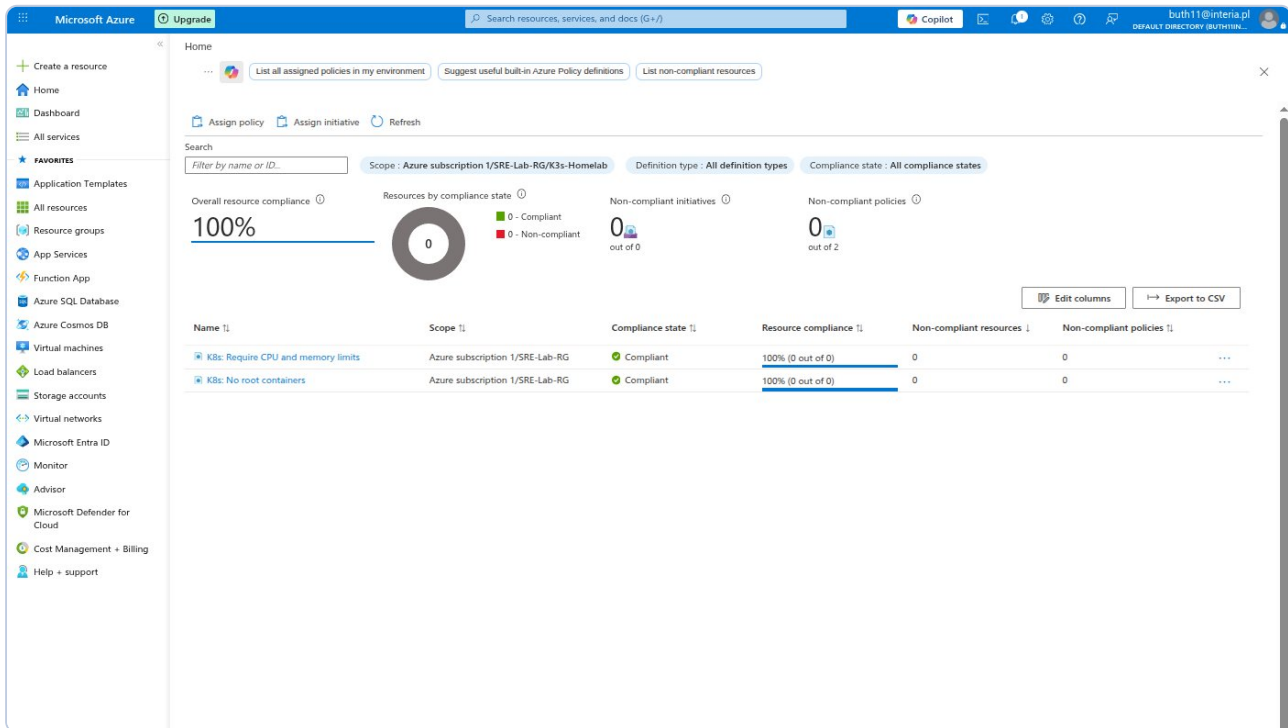
Outputs:
action_group_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/actionGroups/SRE-Lab-Alerts"
budget_name = "SRE-Lab-Budget"
log_analytics_workspace_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.OperationalInsights/workspaces/sre-lab-workspace-5plwbb"
log_analytics_workspace_key = <sensitive>
resource_group_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG"
resource_group_name = "SRE-Lab-RG"
subscription_id = "b023e8b8-4446-4d4f-9fcf-d20a6c1fa093"
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$
```

Terraform Apply complete: 3 added, 1 changed, 0 destroyed.



Azure Portal: SRE-Lab-Alerts, sre-lab-oomkill-alert, sre-lab-workspace in SRE-Lab-RG.

Chapter 10: Azure Policy



Azure Policy: K8s No root containers - Compliant, K8s Require CPU/memory limits - Compliant, 100%.

Chapter 11: GitHub

The screenshot shows the GitHub repository page for 'sre-homelab-azure-arc' by user 'buth11'. The repository is public and has 1 branch (main) and 0 tags. The commit history shows an initial commit 'K3s + Azure Arc SRE Homelab' made 8 minutes ago. The file list includes folders 'docs', 'k8s', 'monitoring', 'scripts', and 'terraform', and files '.gitignore' and 'README.md'. The README file is selected, showing the title 'SRE Homelab: Azure Arc Hybrid Cloud Lab' and a description: 'A Proof-of-Value project demonstrating hybrid cloud orchestration, infrastructure-as-code, observability, and FinOps practices using K3s Kubernetes and Microsoft Azure Arc.' The right sidebar shows the repository's statistics: 0 stars, 0 forks, and 0 watchers. The 'Languages' section shows a bar chart with 'Shell' at 58.4% and 'HCL' at 41.6%.

GitHub repository page for **sre-homelab-azure-arc** (Public) by **buth11**.

Repository details: 1 Branch (main), 0 Tags, 1 Commit (Initial commit: K3s + Azure Arc SRE Homelab, 8 minutes ago).

Files and folders:

- docs
- k8s
- monitoring
- scripts
- terraform
- .gitignore
- README.md

README content:

SRE Homelab: Azure Arc Hybrid Cloud Lab

A Proof-of-Value project demonstrating hybrid cloud orchestration, infrastructure-as-code, observability, and FinOps practices using K3s Kubernetes and Microsoft Azure Arc.

Overview

Repository statistics:

- Readme
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases: No releases published. [Create a new release](#)

Packages: No packages published. [Publish your first package](#)

Contributors: 1 contributor (buth11)

Languages: Shell 58.4%, HCL 41.6%

GitHub repo sre-homelab-azure-arc - public, Shell 58.4%, HCL 41.6%.

Chapter 12: Postmortem POST-001

Field	Value
Severity	P2 - Degraded cluster capacity
Duration	Under 7 minutes
Root Cause	Hostname typo: ks3-worker1 instead of k3s-worker1
Status	Resolved - scripts updated with regex validation

Chapter 13: Results and Conclusions

Metric	Value
Cluster nodes	3 / 3 Ready
Running pods	29
CPU utilisation	avg 2.20% / max 4.84%
RAM utilisation	avg 12.9% / max 14.48%
SLO Availability (24h)	100%
Azure cost	0.00 USD
Terraform resources	5
Security policies	2 Compliant
Issues resolved	6 (ISSUE-001 to ISSUE-006)

Chapter 14: Troubleshooting Log — Issues Encountered During Deployment

Every production environment generates problems. This chapter documents real errors encountered while building this lab, their root causes, and the solutions applied. From an SRE perspective, this is one of the most valuable chapters — it demonstrates diagnostic thinking and the ability to resolve unexpected problems systematically.

ISSUE-001

KQL: Failed to resolve scalar expression CPUCapacityNanoCores

Component	Azure Monitor Log Analytics / KQL
When	First KQL query after AMA installation
Severity	Low — no data returned, cluster unaffected

Symptom:

project operator: Failed to resolve scalar expression named
CPUCapacityNanoCores

Root Cause: The newer AMA (Azure Monitor Agent v3.x) stores CPU and RAM metrics under different column names than documentation suggests for older omsagent versions. The column CPUCapacityNanoCores does not exist in this workspace schema version.

Diagnosis: Ran an exploratory query: `KubeNodeInventory | getschema` — displayed available columns and confirmed absence of CPUCapacityNanoCores.

Resolution: Simplified the query to columns that actually exist in this schema version: Computer, Status, KubeletVersion. CPU/RAM metrics are available through the Perf table instead of KubeNodeInventory.

SRE Lesson: Always verify the table schema before writing queries on a new workspace. Microsoft documentation describes schemas for different agent versions — verify which version is deployed.

Component	Azure CLI / az policy assignment create
When	Assigning K8s: Require CPU and memory limits policy
Severity	Low — command rejected, policy not assigned

Symptom:

(MissingPolicyParameter) The policy assignment k8s-resource-limits is missing the parameter(s) cpuLimit,memoryLimit as defined in the policy definition e345eccc version 9.3.0.

Root Cause: Policy definition e345eccc version 9.3.0 introduced mandatory parameters cpuLimit and memoryLimit that earlier versions did not require.

Resolution: Added required parameters to the CLI command:

```
--params {"cpuLimit": {"value": "500m"}, "memoryLimit": {"value": "512Mi"}}
```

SRE Lesson: Azure policy definition versions can change. Before automating policy assignment in CI/CD pipelines, always check the current definition version and its required parameters.

Component	Terraform / azurerm_resource_group
When	terraform apply — first run
Severity	Medium — apply aborted, 2 of 6 resources not created

Symptom:

Error: A resource with the ID .../resourceGroups/SRE-Lab-RG already exists – to be managed via Terraform this resource needs to be imported into the State.

Root Cause: Resource Group SRE-Lab-RG was previously created manually by the `az connectedk8s connect` command. Terraform had no knowledge of its existence in its state file — it tried to create it and hit a conflict with the existing Azure resource.

Diagnosis: Classic brownfield deployment problem. Terraform requires importing existing resources into state before managing them.

Resolution: `terraform import azurerm_resource_group.sre_lab /subscriptions/b023e8b8-.../resourceGroups/SRE-Lab-RG` After import: regenerate plan and re-apply.

SRE Lesson: In brownfield environments, always start with `terraform import` for resources that already exist. Alternatively, use terraformer to automatically import existing infrastructure into HCL code.

Component	Terraform state management
When	terraform apply tfplan after terraform import
Severity	Low — informational error, easy to resolve

Symptom:

Error: Saved plan is stale. The given plan file can no longer be applied because the state was changed by another operation after the plan was created.

Root Cause: A Terraform plan is a snapshot of state at the moment of generation. The `terraform import` operation changed the state file — the plan generated before the import is no longer consistent with the current state.

Resolution: Generate a new plan after import: `terraform plan -out=tfplan` then `terraform apply tfplan`.

SRE Lesson: Any operation that modifies state (import, taint, state mv) invalidates existing plans. In CI/CD pipelines, plan and apply should be in the same session without interruption for manual state operations.

Component	Azure Arc / azuremonitor-containers extension
When	After cluster virtual machine restart
Severity	Medium — no telemetry to Azure Monitor for ~5 minutes

Symptom: After VM startup, the Azure portal showed the cluster as Connected but the Extensions tab showed "No extensions installed". Container Insights data stopped flowing.

Root Cause: The azuremonitor-containers extension was installed in the previous session but did not survive restart intact — AMA agents required re-registration after the cluster identity certificate was regenerated.

Resolution: Reinstalled the extension with a single command: `az k8s-extension create --name azuremonitor-containers ...` AMA pods (ama-logs) returned to Running state within 37 seconds.

SRE Lesson: Arc extensions are managed by extension-manager which should auto-recover them. The issue suggested the identity certificate expired or was regenerated on restart. In production, configure an alert for missing AMA agents.

Component	K3s / etcd / node registration
When	Cluster provisioning — joining k3s-worker1
Severity	P2 — cluster at 2/3 capacity for 7 minutes

Symptom:

```
FATA[0034] Node password rejected, duplicate hostname or contents of /  
etc/rancher/node/password may not match server
```

Root Cause: Hostname typo during VM provisioning: ks3-worker1 instead of k3s-worker1 (transposed characters). etcd registered the node under the wrong identity. Attempting to rename via hostnamectl triggered etcd Split-Brain protection.

Resolution: 1. `kubectl delete node ks3-worker1` — remove stale etcd entry 2. `sudo hostnamectl set-hostname k3s-worker1` — fix hostname 3. `sudo rm -rf /etc/rancher/node/` — clear agent identity 4. `sudo systemctl restart k3s-agent` — re-register with correct identity Resolution time: under 7 minutes.

Preventive Action: install-k3s-worker.sh updated with preflight check: `if [[ ! "$HOSTNAME" =~ ^k3s-worker[0-9]+$ ]]; then die "Invalid hostname"; fi` — eliminates this entire error class.

Each issue above represents real production troubleshooting. SRE resolves problems methodically: 1) Identify the symptom, 2) Form a hypothesis, 3) Verify the hypothesis, 4) Apply the fix, 5) Document and prevent recurrence.